

DbContext Class in Entity Framework Core

In this article, I will discuss the **DbContext Class in Entity Framework Core**. Please read our previous article, which discusses [how to install Entity Framework Core in .NET applications](#). We will work with the example we created in our previous article and discuss How to Install Entity Framework Core in our .NET Application.

What is the DbContext Class in Entity Framework Core?

The DbContext class is a core component of Entity Framework Core (EF Core) that acts as a bridge between your application's domain (entities) classes and the underlying database. It manages database connections, performs CRUD (Create, Read, Update, Delete) operations, manages transactions, and tracks changes to entities. The DbContext class is responsible for interacting with the database using the configured database provider (e.g., SQL Server, SQLite, etc.).

What are the Tasks Performed by DbContext in Entity Framework Core?

DbContext performs several essential tasks in EF Core. They are as follows:

- **Entity Tracking:** It tracks the state of entities (Added, Modified, Deleted, or Unchanged) and keeps a record of changes that need to be persisted in the database.
- **Database Interactions:** Executes queries, commands, and transactions against the database using the configured provider.
- **Query Execution:** It translates LINQ queries into SQL and executes them against the database.
- **Change Detection:** Identifies changes in the entities and prepares the corresponding SQL commands for execution. That means it generates and executes SQL commands to insert, update, or delete data based on changes to entity objects.
- **Transaction Management:** Manages transactions for multiple operations, ensuring that changes are committed or rolled back as needed.
- **Lazy Loading:** Enables lazy loading of related entities if configured, meaning related data is loaded only when accessed.
- **Caching:** Provides first-level caching of entities during a single DbContext instance's lifetime, reducing the need for repeated database calls for the same entity.
- **Concurrency Control:** Handles concurrency conflicts using optimistic concurrency, allowing for handling multiple users making changes simultaneously.
- **Managing Relationships:** It manages relationships between entities, including the loading and updating related data.

Example to Understand DbContext Class in Entity Framework Core:

Let us understand the need and use of the DbContext Class in EF Core with an example. At the project's root directory, create a folder named **Entities**, where we will create two class files named **Student.cs** and **Branch.cs**.

Creating Student Entity:

So, create a class file named **Student.cs** within the Entities folder and copy and paste the following code. As you can see, we are creating the Student class with a few scalar properties and one Reference Navigation property called Standard. This will make the relationship between Student and Branch entities one-to-one, i.e., one Student can belong to a single Branch. In the next step, we are going to create the Standard entity.

```
namespace EFCoreCodeFirstDemo.Entities
{
    public class Student
    {
        public int StudentId { get; set; }
```

```

        public string? FirstName { get; set; }
        public string? LastName { get; set; }
        public DateTime? DateOfBirth { get; set; }
        public string? Gender { get; set; }
        public string? Email { get; set; }
        public string? PhoneNumber { get; set; }
        public DateTime EnrollmentDate { get; set; }
        // Navigation property representing the Branch the student is
        enrolled in
        public virtual Branch? Branch { get; set; }
    }
}

```

Student Entity Properties:

- **StudentId**: The Student Unique Identifier. It is the Primary key in the Students table.
- **FirstName & LastName**: These are primary identifiers for a student.
- **DateOfBirth**: Helps calculate the age or any date-related information.
- **Gender**: Indicates the gender of the student.
- **Email & Phone Number**: These are essential for communication.
- **EnrollmentDate**: Captures when the student was enrolled in the institution.
- **Branch**: Represents a reference to the Branch entity, establishing a relationship between the Student and the Branch.

Creating Branch Entity:

Next, create another class file named **Branch.cs** within the Entities folder and copy and paste the following code. As you can see, we are creating the Branch class with a few scalar properties and one collection Navigation property called Students. This makes the relationship between Standard and Student entities one-to-many, i.e., one Branch can have multiple Students.

```

namespace EFCoreCodeFirstDemo.Entities
{
    public class Branch
    {
        public int BranchId { get; set; }
        public string? BranchName { get; set; }
        public string? Description { get; set; }
        public string? PhoneNumber { get; set; }
        public string? Email { get; set; }
        // Collection navigation property representing the students enrolled
        in the branch
        public ICollection<Student>? Students { get; set; }
    }
}

```

Branch Entity Properties:

- **BranchId**: The Branch Unique Identifier. It is the Primary key in the Branches table.
- **BranchName**: The name of the branch (e.g., "CSE", "ETC", "Mechanical", etc.).
- **Description**: A brief description of the branch.

- **Phone Number & Email:** These fields provide contact details of the branch.
- **Students:** A collection that holds all students belonging to a particular branch, establishing a one-to-many relationship.

We have completed creating the initial domain classes for our application. As we progress in this course, we will add more domain classes to this example.

How Do We Create a DbContext Class in Entity Framework Core?

To create a DbContext class in Entity Framework Core, we need to define a custom class that derives from the **DbContext** class and includes properties of type `DbSet<T>` for each entity class we want to model in the database. The DbContext class is present in **Microsoft.EntityFrameworkCore** namespace. So, let us see how to create a DbContext class that includes the Student and Branch entities.

So, create a class file within the **Entities** folder named **EFCoreDbContext.cs** and copy and paste the following code into it. You can give any name for your context. However, the class should and must be derived from the DbContext class, which exposes DbSet properties for the types we want to be part of the model, i.e., Student and Branch domain classes. As per the Microsoft recommendations, we have given the property name in the plural form of the entity name, like Students and Branches.

```
// Import the Entity Framework Core namespace to access DbContext and other EF
Core functionalities.
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    // EFCoreDbContext class inherits from DbContext, which is the primary class
    for interacting with the database using EF Core.
    public class EFCoreDbContext : DbContext
    {
        // Constructor that accepts DbContextOptions<EFCoreDbContext> as a
        parameter.
        // The options parameter contains the settings required by EF Core to
        configure the DbContext,
        // such as the connection string and provider.
        public EFCoreDbContext(DbContextOptions<EFCoreDbContext> options)
            : base(options) // The base(options) call passes the options to the base
        DbContext class constructor.
        {
        }
        // OnConfiguring is an override method that allows configuring the
        DbContext options,
        // like setting the database provider and connection string.
        protected override void OnConfiguring(DbContextOptionsBuilder
        optionsBuilder)
        {
            // Configure the database provider and connection string.
            // UseSqlServer method configures the DbContext to use SQL Server
            the database provider.
            // The provided connection string specifies the server, database
            name, and credentials.
            // Replace "Server=YourServerName;Database=YourDatabaseName;User
            Id=YourUsername;Password=YourPassword;"
```

```

        // with your actual SQL Server details.

optionsBuilder.UseSqlServer("Server=YourServerName;Database=YourDatabaseName;Initial Catalog=YourDatabaseName;User Id=YourUsername;Password=YourPassword;");
    }
    // DbSet<Student> Students represents a table in the database
corresponding to the Student entity.
    // EF Core uses DbSet<TEntity> to track changes and execute queries
related to the Student entity.
    public DbSet<Student> Students { get; set; }
    // DbSet<Branch> Branches represents a table in the database
corresponding to the Branch entity.
    // Similar to the Students DbSet, this property is used by EF Core to
track and manage Branch entities.
    public DbSet<Branch> Branches { get; set; }
}
}

```

Explanation of the Code:

- **DbContext:** The main class in EF Core for interacting with the database. It manages entity objects during runtime and handles database connections.
- **Constructor:** The constructor initializes the DbContext using the options provided, which typically include the database provider and connection string.
- **OnConfiguring Method:** This method configures the DbContext, such as setting the database provider and connection string. It is useful if you don't provide configuration externally.
- **DbSet<TEntity>:** These properties represent collections of entities that EF Core tracks. They correspond to tables in the database, and each entity type (like Student and Branch) maps to a DbSet.

Note: We have included two model classes as DbSet properties, and the entity framework core will create two database tables for the above two model classes with the required relationships.

What are the Methods Provided by DbContext in Entity Framework Core?

The DbContext class provides various methods to manage and interact with the database in EF Core. Some of them are as follows:

- **Add/Attach/Update/Remove:** Methods for adding, attaching, updating, or removing entities from the context.
- **SaveChanges / SaveChangesAsync:** Persists changes to the database.
- **Find / FindAsync:** Finds an entity by its primary key.
- **Entry:** Provides access to change tracking information for a specific entity.
- **OnConfiguring:** Allows configuring the context options (e.g., connection string) when not using dependency injection.
- **AddRange/ AddRangeAsync:** This function adds a collection of new entities to DbContext with the Added state and starts tracking them. The new entities will be inserted into the database when SaveChanges() or SaveChangesAsync() is called.

To connect to a database, we need the database connection string. So, in the next article, I will discuss where to define the [Connection String in Entity Framework Core](#) and how to use it to interact with the SQL Server Database. In this article, I explain the need and use of the **DbContext Class in Entity Framework Core**. I hope you enjoy this Entity Framework Core DbContext Class article.